Materializer receipt journal consumer

[pcc:unverified] T-1747. Source and deterministic software unit tests, 2026-11-07.

Implemented a reusable, read-only Python standard-library verifier for journal-prefix/v1 and inline-final-and-journal-prefix/v1 receipts. The producer contract is materializer source SHA256 085234b563a2c766040706ed7d805b2971074efba14da022265a6aa6db33ca63 and its published independent review at 2f0bfc0221.

The reader verifies exact prefix byte length, SHA256, record count, JSONL boundaries and object records. It rejects duplicate JSON keys, nonfinite numbers, Boolean or negative counts, unknown status/storage, symlinks, parent traversal, nonregular files, oversized inputs and files changing during a read. Every directory component is opened with O_NOFOLLOW. The journal basename is fixed.

Passed final receipts require exact typed equality between inline records and the entire committed journal, with no unreferenced tail. Failed final receipts may retain additional inline records or unreferenced bytes, but their committed prefix must match. Intermediate records always remain incomplete even if reported_status is passed. Missing pointers acknowledge no prefix; they do not establish that no children existed. Legacy inline records receive a separate weaker result with no journal verification. Final status incomplete is rejected because the producer final ABI uses only failed or passed.

Observed software checks: 19 unittest methods passed in 0.006 seconds reported by unittest, under a 25-second process timeout. Adversarial fixtures use temporary directories; they exercise hash, count, truncation, malformed JSON, type mismatch, tail and symlink rejection, failed append representation, empty prefixes and legacy evidence. This is software-test duration, not a materializer performance measurement. Earlier 18-method run passed before independent review identified incomplete-final suffix acceptance; the correction and regression fixture are retained in the review record.

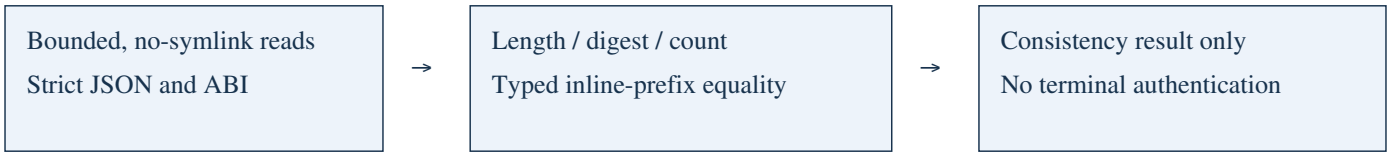
Input limits: receipt 16 MiB, committed prefix 16 MiB, physical journal 64 MiB and 100,000 records. These conservative limits are explicit API policy; exceeding them rejects evidence rather than truncating silently. Larger future receipts require a separately reviewed adjustment.

The reader reports storage consistency only, named storage_complete, storage_failed or storage_incomplete. It explicitly returns cleanup_semantics_verified=false; cleanup errors or receipt error arrays are not interpreted as successful cleanup. It never authenticates terminal ownership, authorizes execution, examines credentials, controls jobs or imports the materializer. Status and storage assertions remain untrusted evidence even when internally consistent. Concurrent cross-file changes are not an atomic filesystem snapshot; callers must obtain stable archived receipts and separately authenticate process ownership. File metadata stability checks detect ordinary concurrent writes, not a malicious actor able to restore metadata or rewrite multiple files coherently.

Next dependency: independent source/fixture review, followed by integration into the bounded materializer admission workflow. Actual filesystem durability, fault recovery, runtime completion and scientific cohort evaluation remain separate work. No compiler, build, scientific worker or campaign launched for this packet.

Reproduce software fixtures from this directory: timeout 25s python3 -B -m unittest -v test_receipt_reader.py. No third-party package is needed for the reader or tests. The report renderer uses already-installed PyMuPDF only.

Read-only receipt consistency checks (source and software fixtures)



Intermediate → incomplete; failed final → prefix plus possible extra evidence
Passed final → exact full journal; legacy → weaker inline-only evidence

Receipt verifier diagram; no materializer runtime measurements.